# Finlay Fordham (730038662) and Angelo Thind (730041272) Software Development CA:

# Test Design:

- We used JUnit 5.x framework
- We decided to create a Test file for each class we created. For instance, for the hand class file we would have a corresponding handTest file. This follows for every other class except CardCollections, as it is an interface which is implemented by other classes and so doesn't have any implemented methods that can be tested.
- Inside each of these files we used Junit to create a unit test for every method in every class.
- The following document will detail the design of each and what they test for.
- All tests for methods are named in the fashion methodNameTest.

# UNIT TESTS:

- These are tests for each individual method in each class which are highly specific to create a strong functional base code which runs smoothly and handles errors effectively.

## Card:

The base class only has one method "getValue" and so the test file only contains one unit test. This test will check if a card can hold a variety of values, including potential edge cases such as 0, negative integers and large numbers.

## Hand:

The base class contains 3 methods, namely, getCards(), addCard() and removeCard(). Firstly, the test for getCards() and addCard() cannot be completed without using the other and so we thoroughly tested both to make sure neither has issues. For getCards() this involved, checking it can correctly return a simple hand, a hand with more complex cards such as potential edge cases, a hand with multiple of the same card, and handling when a request is made for an empty hand. For addCard(), we will add a variety of cards and validate the output using assertEquals with getCards() and our expected result. Finally, for removeCard() we will test if removing from a specific index does as expected, for a variety of positions that could potentially cause issues (first card, last card, cards between), and handling what to do when removing a card from an empty hand.

## Deck:

The base class contains 4 methods, namely, getDeckId(), addCard(), getDeck(), removeCard(). Firstly, similarly to Hand getDeck() and addCard() cannot be tested without using the other, we tackled this in the same way as previously. For getDeckId() we will test with a variety of id values that may have potentially been edge cases. Secondly, for addCard() we will test if you can add a variety of card values to a deck, check if multiple decks can be added to without interfering with each other, and see if the same card can be added to the same deck multiple times. Thirdly, for getDeck() we will check basic functionality, check if different cards with matching number can be stored, and if you can get an empty deck / if it is handled well.

## Pack:

The base class contains 3 methods, addCard(), getPack(), and removeCard(). Firstly, similarly to Deck and Hand, you cannot test addCard() without also testing getPack(), we handled this in the same way as previously mentioned. To test addCard(), we will test if you can add a variety of card values (including potential edge cases) to the pack. Secondly, for getPack() we will check if you can get the pack when it is made up of simple cards, more complex cards, with the same card in a pack multiple times (and same value by proxy). Finally, for removeCard() we will check for basic functionality and how removing a card from an empty pack is handled.

## Player:

The base class contains 7 methods, getHand(), getDiscardDeck(), getPreference(), getDrawDeck, draw(), discard() and, haswon(). Firstly, to test getHand() we will create a mock player which is holding the deck, then we will check if getHand() successfully returns the same hand that was passed to it using assertions. Secondly, getDiscardDeck() will be tested by checking if the deck assigned at instantiation matches the deck returned by the method. Thirdly, for getPreference() we will instantiate a player with a preference value and then check if getPreference() returns the same value back to us. Fourthly, getDrawDeck() will be tested in a similar method to previous methods, we will create a player with a set drawDeck then check if the returned deck matches the deck we gave the player. Draw() will be tested by creating a “clone” object of the hand the player has using the methods Hand has and then comparing the output of getHand() after draw has been run. Discard() will be tested by checking if the cards discarded are contained in a list of possible random cards to discard (not containing the preference), this will also be tested in edge cases such as when there are no favourable cards, there is no need to test when all cards are favourable as the game would have already ended. Finally, the hasWon() method will be tested by creating mock player objects and hand objects in various states (winning hand, losing

hand, close to winning hand), these will each be checked by an assertion to confirm the correct result is returned.

# GROUP TESTS:

- These tests are tests which combine groups of previous unit tests in the main file methods, this helps to smooth out problems with passing data between methods and find logical problems in the main code.

## CardGame:

The base class contains 11 methods not including main, all these methods are private and so we will use reflection to change the accessibility of methods we would like to test. These are createPack(), validatePack(), playerDiscards(), playerDraws(), hasPlayerWon(), logCurrentHand(), logDrawnCard(), logDisCard, logGameEnd() and, endGame(). Firstly createPack() will be tested by reading directly from the pack file and creating a direct list of integers it contains; these will then be compared to the values held on the card objects in the pack made from the same file to see if they match. Secondly, the validatePack() method which is meant to check if the cards contained within a pack are valid (i.e. there is the correct number and amount of each for each player to be to win), will be tested by creating a variety of mock packs which will be passed into the method with an expected result which will be checked using an assertion. Thirdly, playerDiscards will be tested by checking the state of each object involved after the method has been invoked to assert that they have been affected in the intended way, this same method will also be used for playerDraws just with the draw deck not the discard deck. Fourthly, hasPlayerWon() will be tested by creating various mock players in different states (winning, not winning) which will then be passed into the method and asserted to check if the expected output is given, the situation where two players win at once will not be tested as specified in the specification. Next, the logCurrentHand(), logDrawnCard(), logDisCard(), logGameEnd() will all be tested by creating mock objects to emulate the situation they would be used in, then we will invoke them to write to a file what they will log. This file will then be read from and using an assertion we will check if the log is the expected result for each mock situation we have designed. Finally, the endGame() method will be tested by inputting different mock players and game end states to see if it will output the correct message (or not output a message at all) to the terminal. We will then read this message and compare it to the expected message for functional behaviour.